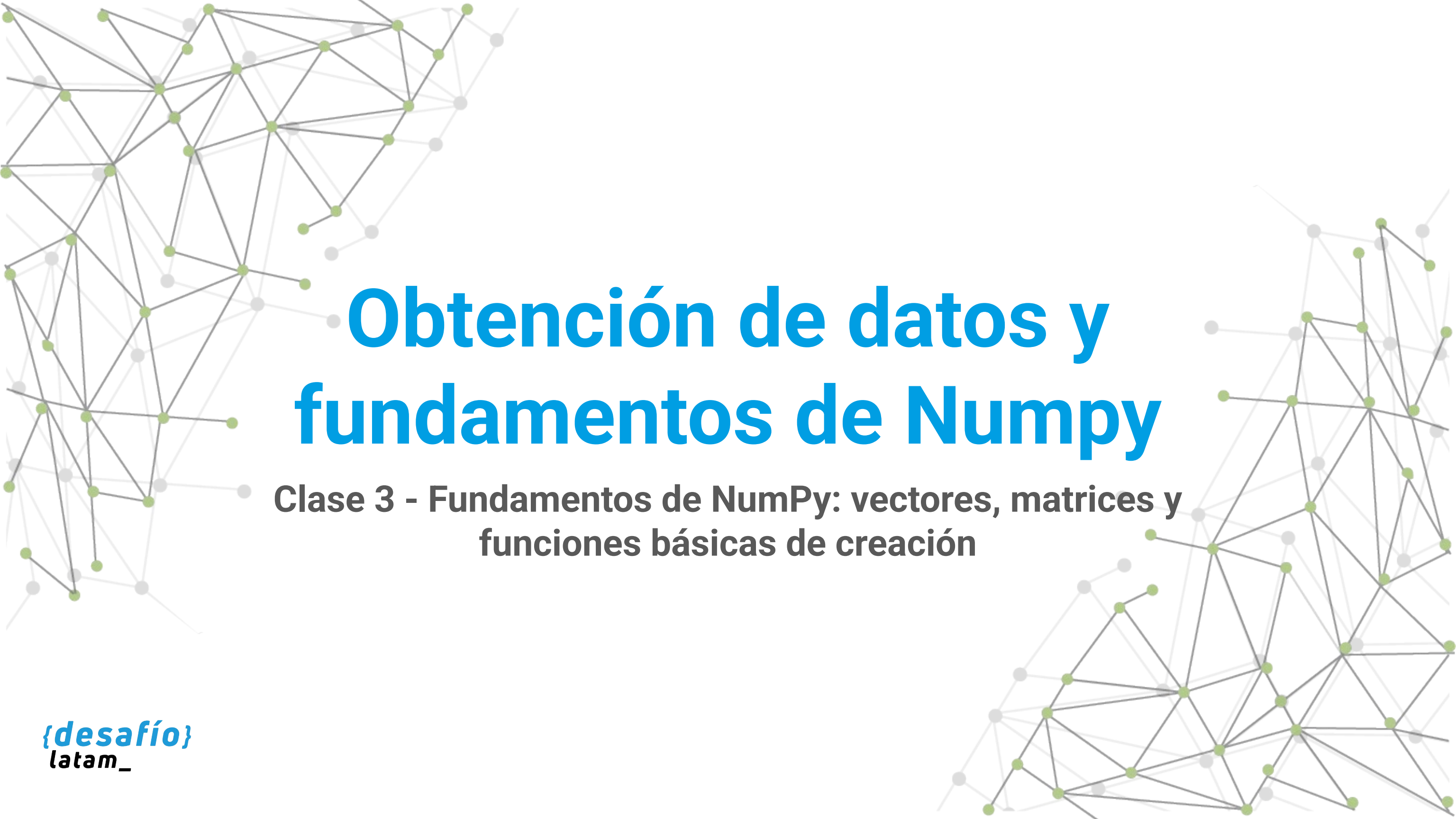
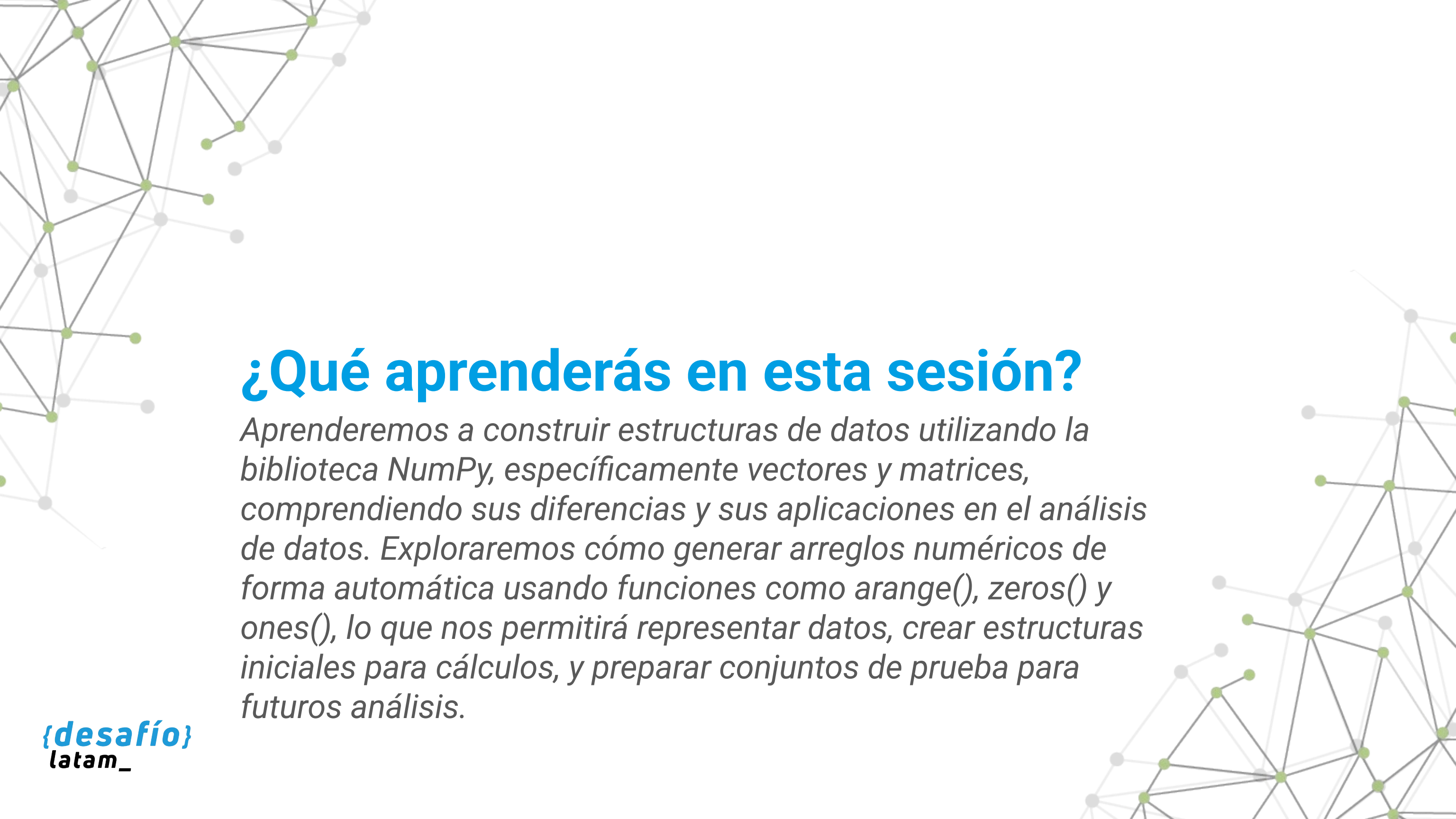




Obtención de datos y fundamentos de Numpy

**Clase 3 - Fundamentos de NumPy: vectores, matrices y
funciones básicas de creación**



¿Qué aprenderás en esta sesión?

Aprenderemos a construir estructuras de datos utilizando la biblioteca NumPy, específicamente vectores y matrices, comprendiendo sus diferencias y sus aplicaciones en el análisis de datos. Exploraremos cómo generar arreglos numéricos de forma automática usando funciones como `arange()`, `zeros()` y `ones()`, lo que nos permitirá representar datos, crear estructuras iniciales para cálculos, y preparar conjuntos de prueba para futuros análisis.

Desafío inicial

Formas parte de un equipo de ingeniería que monitorea dispositivos electrónicos que registran datos en tiempo real. Cada segundo se generan decenas de lecturas de voltaje, temperatura y humedad. Estas mediciones deben almacenarse en estructuras que permitan su análisis inmediato. Sin embargo, no puedes hacerlo manualmente: necesitas estructuras eficientes, rápidas de crear y que puedan ser manipuladas fácilmente con operaciones matemáticas.



Para comenzar, debes construir plantillas y simulaciones básicas con estructuras unidimensionales (vectores) y bidimensionales (matrices), que sirvan como base para organizar, filtrar y calcular con estos datos en etapas posteriores.

¿Cómo podrías estructurar esa información para que sea flexible, reutilizable y lista para el análisis?

Para resolver esta necesidad, deberás aprender a construir estructuras eficientes como vectores y matrices utilizando la biblioteca NumPy, y generar datos automáticamente con funciones como `arange()`, `zeros()` y `ones()`.

`/* Librería NumPy */`

La librería NumPy y su utilidad

¿Por qué usar NumPy en lugar de listas tradicionales de Python cuando trabajamos con datos numéricos?

NumPy (Numerical Python) es una biblioteca fundamental para la ciencia de datos en Python, especialmente diseñada para procesamiento numérico eficiente. Su componente central es el tipo de dato `ndarray`, que permite almacenar arreglos homogéneos (todos los elementos del mismo tipo) en una o más dimensiones y realizar operaciones vectorizadas de forma altamente optimizada.



A diferencia de las listas nativas de Python, que están diseñadas para versatilidad general, NumPy está construido sobre bibliotecas de bajo nivel como BLAS y LAPACK, lo que permite realizar operaciones aritméticas, estadísticas y algebraicas con gran velocidad y eficiencia de memoria.

Esto es crucial cuando se trabaja con grandes volúmenes de datos, como series temporales, simulaciones físicas, modelos predictivos o datos de sensores en tiempo real.



Comparación entre listas de Python y arreglos NumPy

Supongamos que queremos duplicar los valores de una secuencia de números. Con listas tradicionales, usaríamos comprensión de listas:

```
lista = [1, 2, 3]  
doble = [x * 2 for x in lista]
```


Con NumPy, el código se simplifica y optimiza:

```
import numpy as np
arreglo = np.array([1, 2, 3])
doble = arreglo * 2
```

Este tipo de operación directa se denomina vectorización. NumPy aplica la operación a todos los elementos del arreglo de forma simultánea, sin necesidad de recorrerlo con bucles explícitos.

¡Importante! Esto no solo hace el código más limpio, sino también mucho más rápido, especialmente al trabajar con miles o millones de elementos.

Ventajas técnicas de NumPy frente a listas nativas

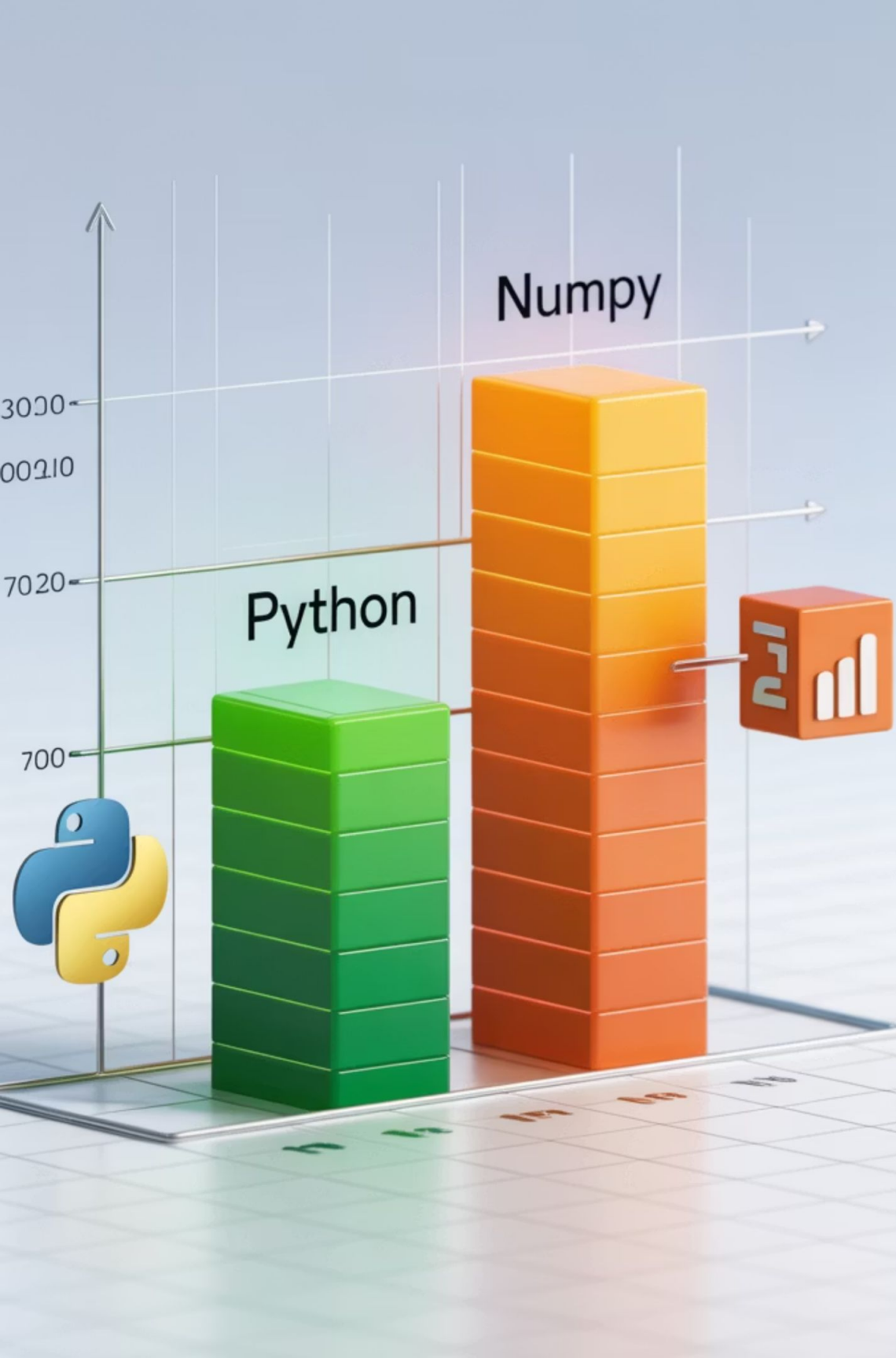
Característica	Listas de Python	Arreglos NumPy
Tipo de datos	Heterogéneo	Homogéneo
Operaciones matemáticas	Requiere bucles	Vectorizadas
Velocidad de ejecución	Lenta en grandes volúmenes	Muy rápida gracias a optimización en C
Consumo de memoria	Alto	Reducido
Funcionalidad científica	Limitada	Alta (funciones algebraicas, estadísticas, etc.)

Ejemplo de rendimiento comparativo (opcional para demo en vivo)

Para ilustrar la diferencia de rendimiento, se puede usar timeit:

```
import timeit
print("Lista:", timeit.timeit('[x*2 for x in range(10000)]', number=1000))
print("NumPy:", timeit.timeit('import numpy as np; np.arange(10000)*2', number=1000))
```

En este ejemplo, NumPy es decenas de veces más rápido que las listas para operaciones simples como la multiplicación escalar.



Aplicaciones comunes de NumPy



Procesamiento de datos científicos y de sensores

NumPy permite procesar eficientemente grandes volúmenes de datos provenientes de experimentos científicos y sensores.



Análisis de imágenes

Cada imagen puede representarse como una matriz NumPy, facilitando su procesamiento y análisis.



Simulaciones numéricas

En física, biología o ingeniería, NumPy permite realizar simulaciones complejas con alta eficiencia.



Preprocesamiento para Machine Learning

NumPy es fundamental para preparar datos antes de alimentarlos a modelos de aprendizaje automático.



Análisis financiero y series temporales

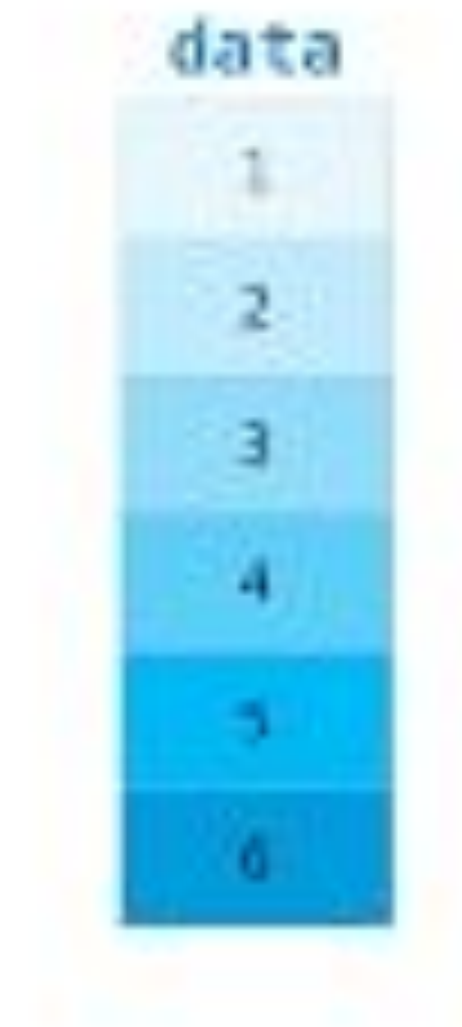
Permite procesar y analizar datos financieros y series temporales con gran eficiencia.

/* Creación de vectores en NumPy */

Creación de vectores en NumPy

¿Cómo representar en NumPy una secuencia de valores unidimensionales que pueda ser manipulada matemáticamente?

En ciencia de datos, es común trabajar con datos unidimensionales: temperaturas, precios, mediciones, puntajes, entre otros. En NumPy, este tipo de estructura se representa mediante un vector, que en términos técnicos es un arreglo unidimensional (ndarray con una sola dimensión).



Creación básica de un vector

Para crear un vector en NumPy, utilizamos la función `np.array()` pasando una lista de valores numéricos:

```
import numpy as np  
vector = np.array([10, 20, 30, 40])
```

Este vector tiene cuatro elementos y representa una única dimensión.





Explorando sus características

Podemos inspeccionar varias propiedades del arreglo creado:

```
print(vector.ndim)
# Devuelve 1 → una dimensión
print(vector.shape)
# Devuelve (4,) → 4 elementos en una dimensión
print(vector.dtype)
# Devuelve el tipo de dato, ej: int64 o float64
```

Esto nos ayuda a comprender su estructura interna y preparar el arreglo para cálculos posteriores.

Operaciones vectorizadas

Una de las principales fortalezas de NumPy es la aplicación de operaciones matemáticas directamente sobre el vector, sin necesidad de bucles:

```
print(vector + 5) # Suma 5 a cada elemento: [15 25 35 45]  
print(vector * 2) # Multiplica por 2: [20 40 60 80]
```

Esto permite realizar transformaciones sobre los datos de forma muy rápida y clara.



Indexación y acceso a elementos

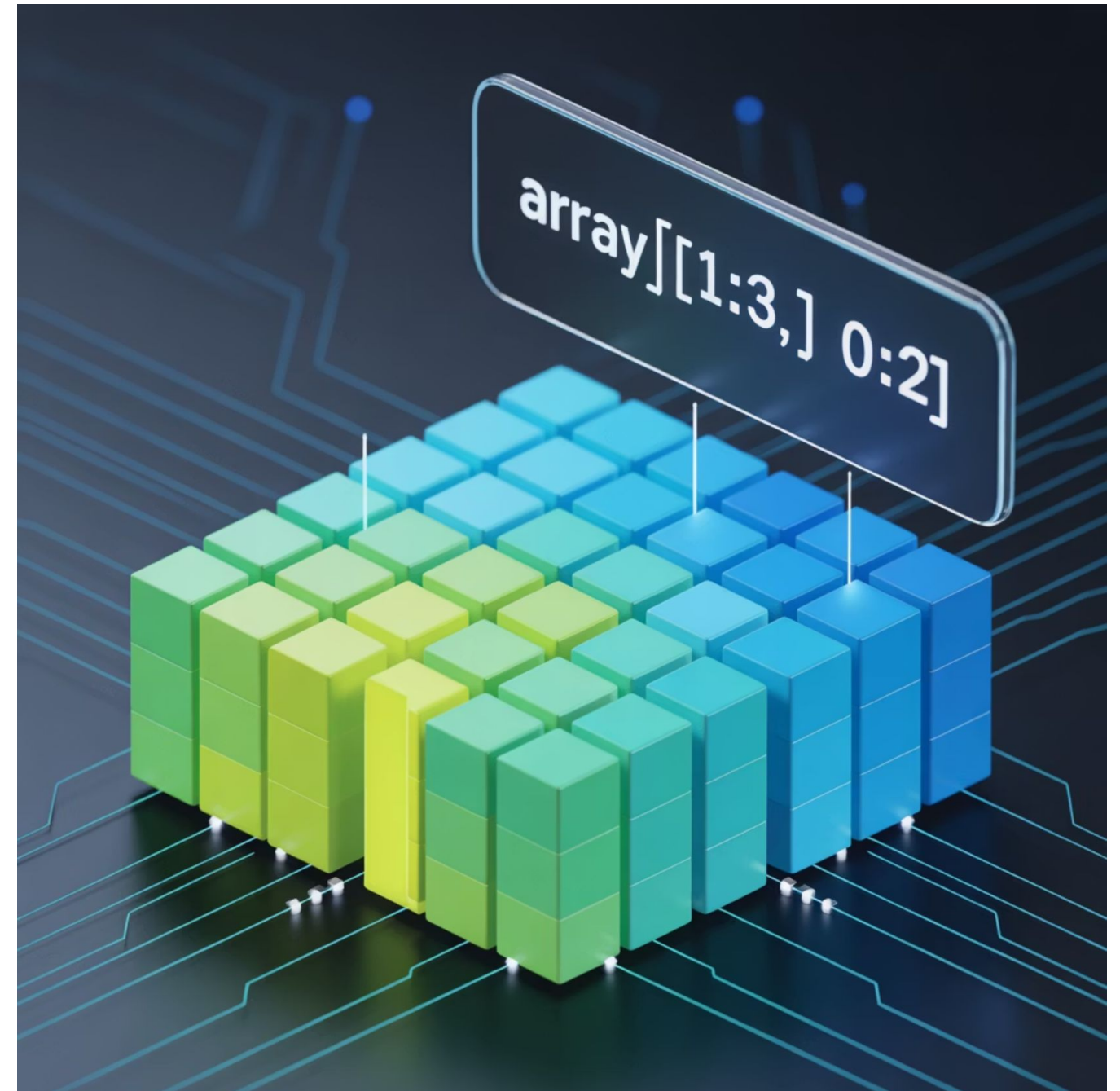
Al igual que las listas de Python, los vectores de NumPy se pueden indexar:

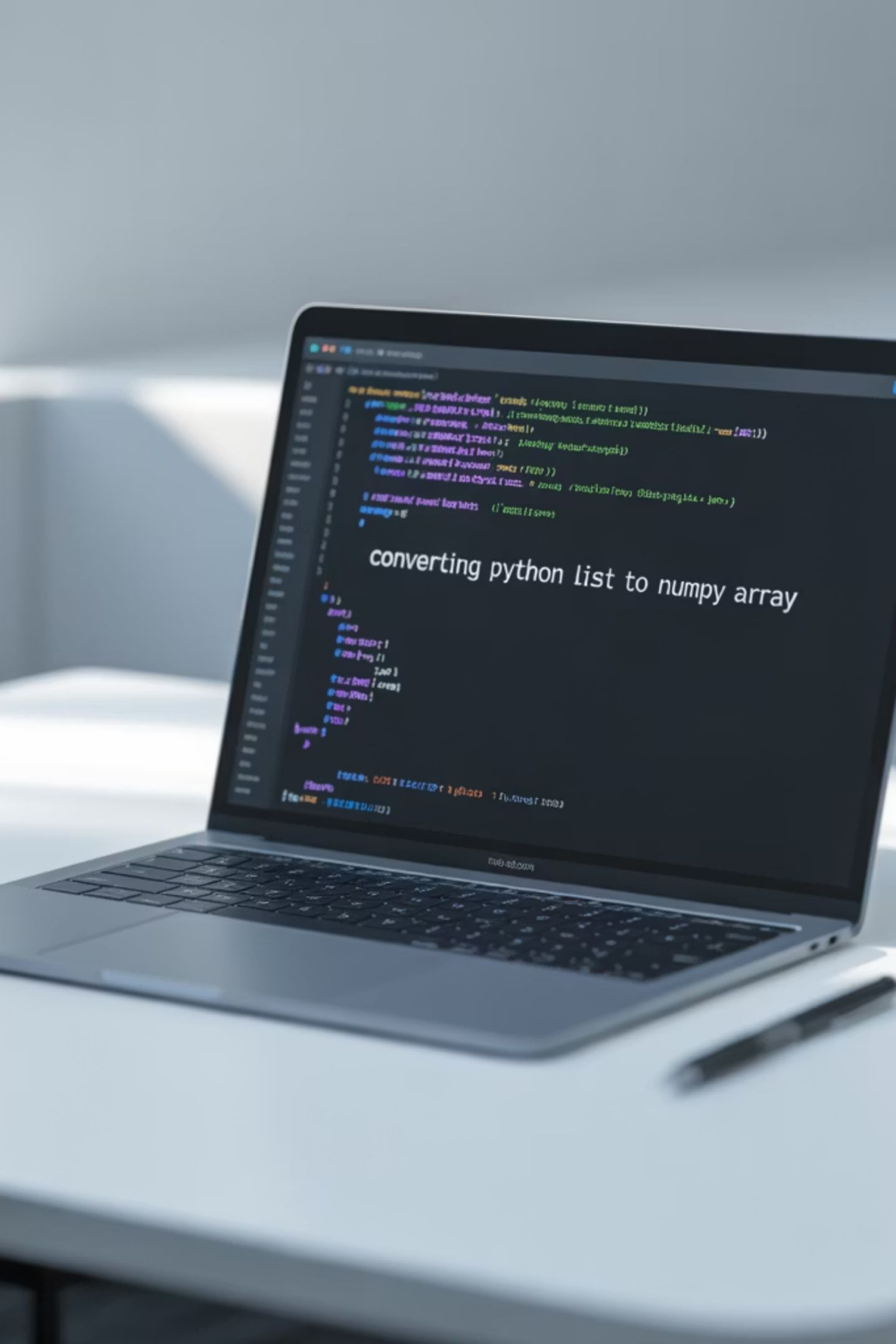
```
print(vector[0])  
# 10  
print(vector[-1])  
# 40 (último valor)
```

También es posible usar rebanado (slicing):

```
print(vector[1:3]) # [20 30]
```

Estas capacidades son muy útiles al analizar subconjuntos de datos o preparar entradas para algoritmos.





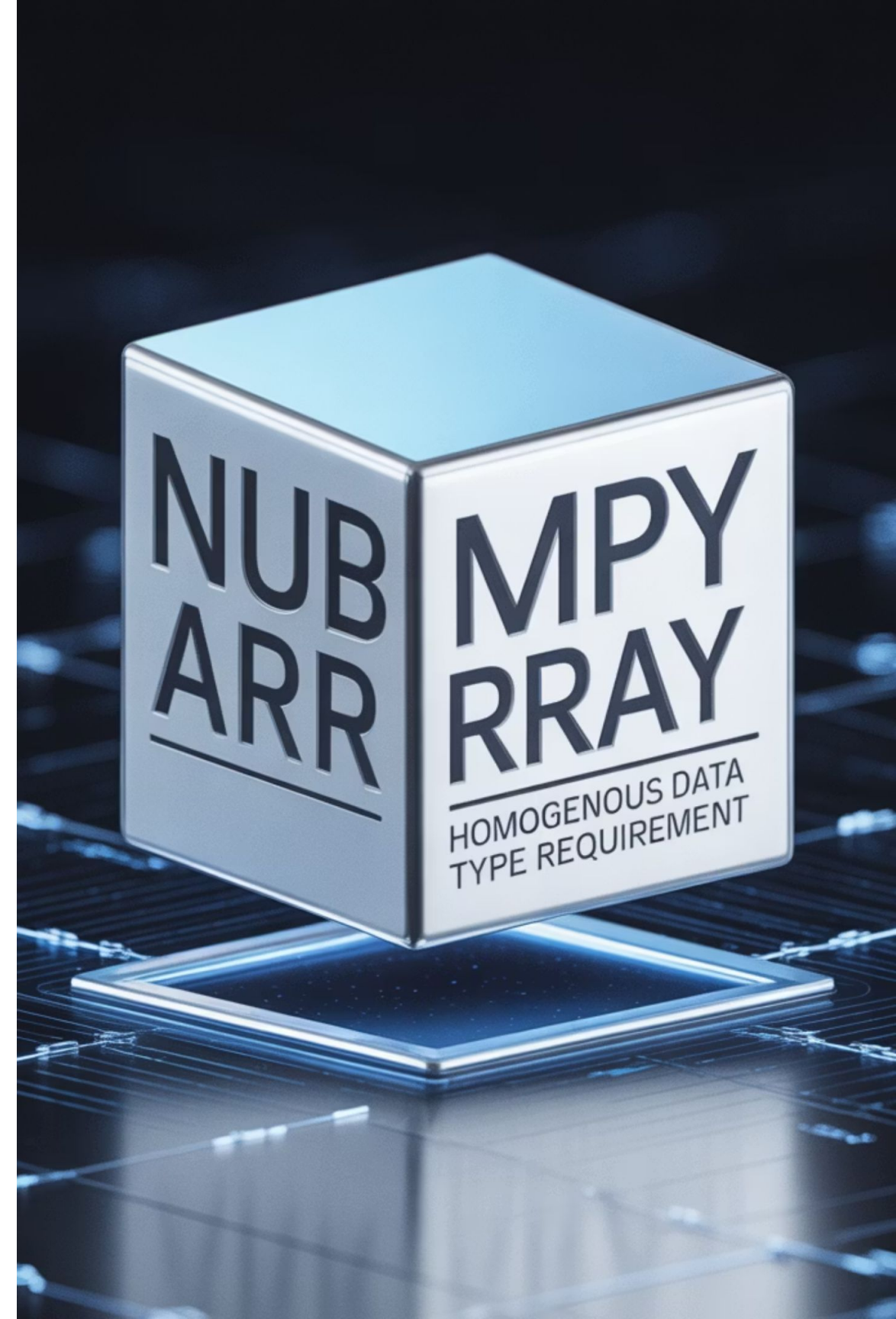
Conversión desde listas existentes

Es común partir con listas tradicionales y convertirlas a NumPy para aprovechar su velocidad y funciones:

```
lista = [5, 10, 15]arreglo = np.array(lista)
```

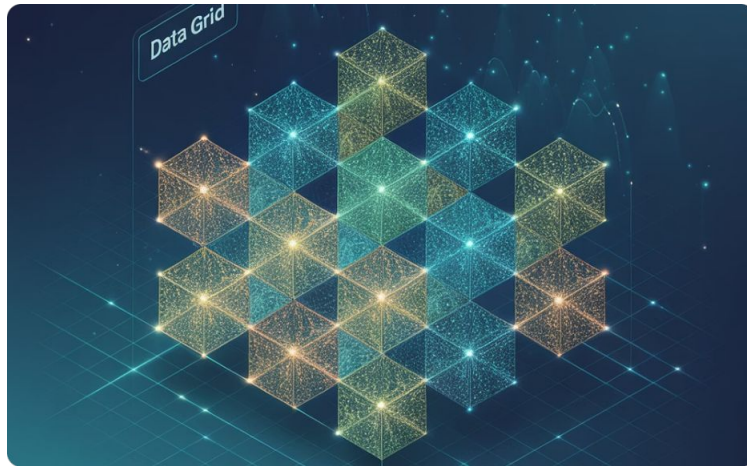

Cuidados al crear vectores

- Todos los elementos deben ser del mismo tipo (int, float, etc.). Si hay mezcla (por ejemplo, ['5', 10]), NumPy convertirá todo a str.
- Las operaciones matemáticas se aplican elemento a elemento, no al vector completo como un bloque.



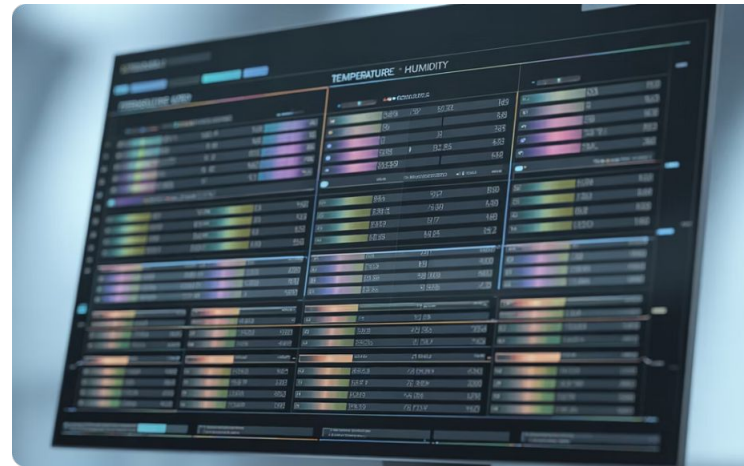
Creación de matrices

¿Cómo puedes organizar múltiples registros de datos en una estructura bidimensional lista para el análisis?



Estructura de matrices

En NumPy, una matriz es simplemente un ndarray de dos dimensiones que organiza los datos en filas y columnas para facilitar cálculos matemáticos y estadísticos.



Observaciones y variables

En ciencia de datos, cada fila representa una observación o instancia (por ejemplo, una medición tomada en un momento específico), y cada columna representa una variable o característica (temperatura, humedad, presión).



De listas a matrices

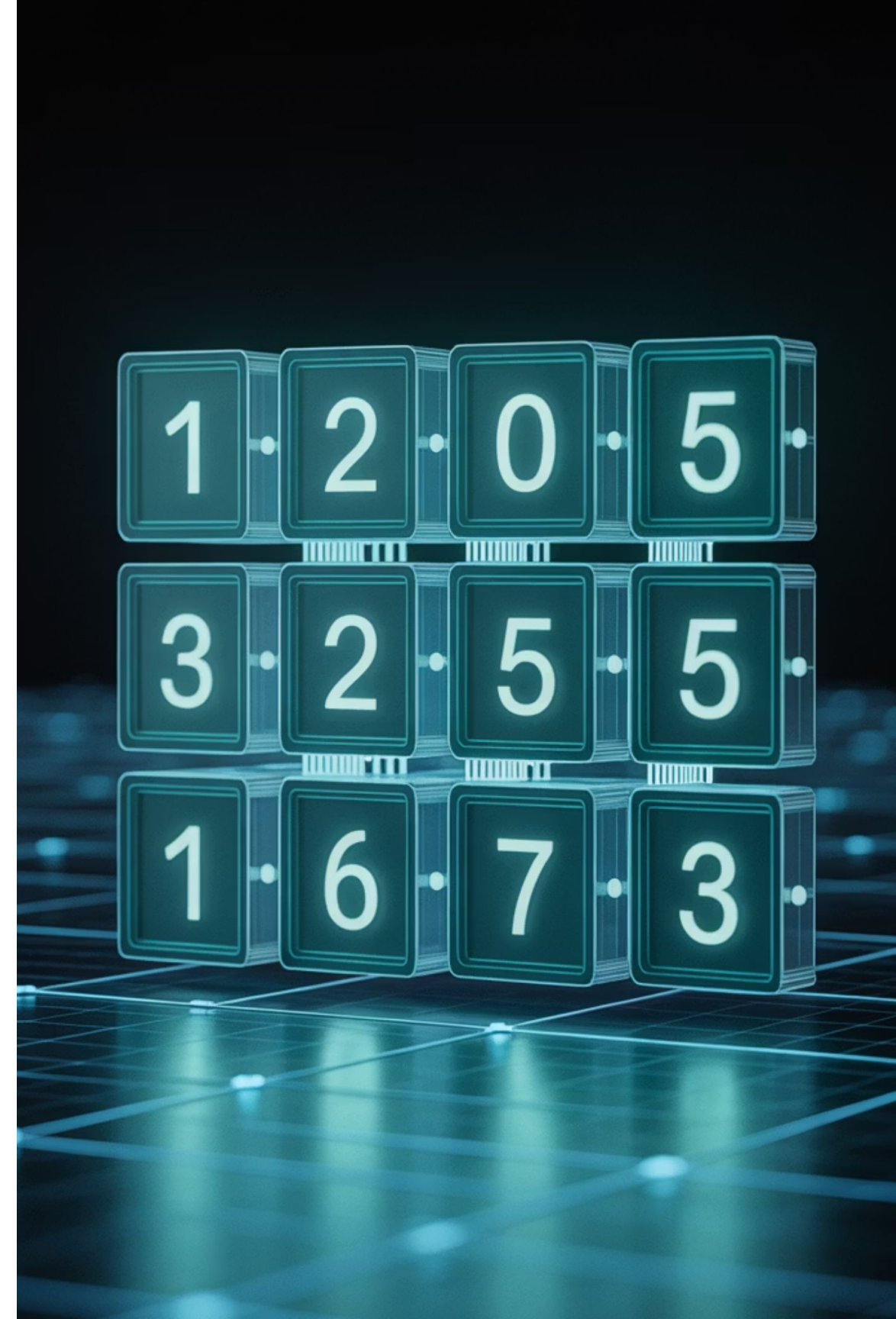
Las matrices se pueden crear fácilmente a partir de una lista de listas en Python, permitiendo transformar datos estructurados en objetos NumPy optimizados para análisis matemático.

Creación de una matriz

```
import numpy as np  
matriz = np.array([[1, 2, 3],[4, 5, 6]])
```

Esta matriz tiene 2 filas y 3 columnas. Visualmente:

```
[[1 2 3] [4 5 6]]
```

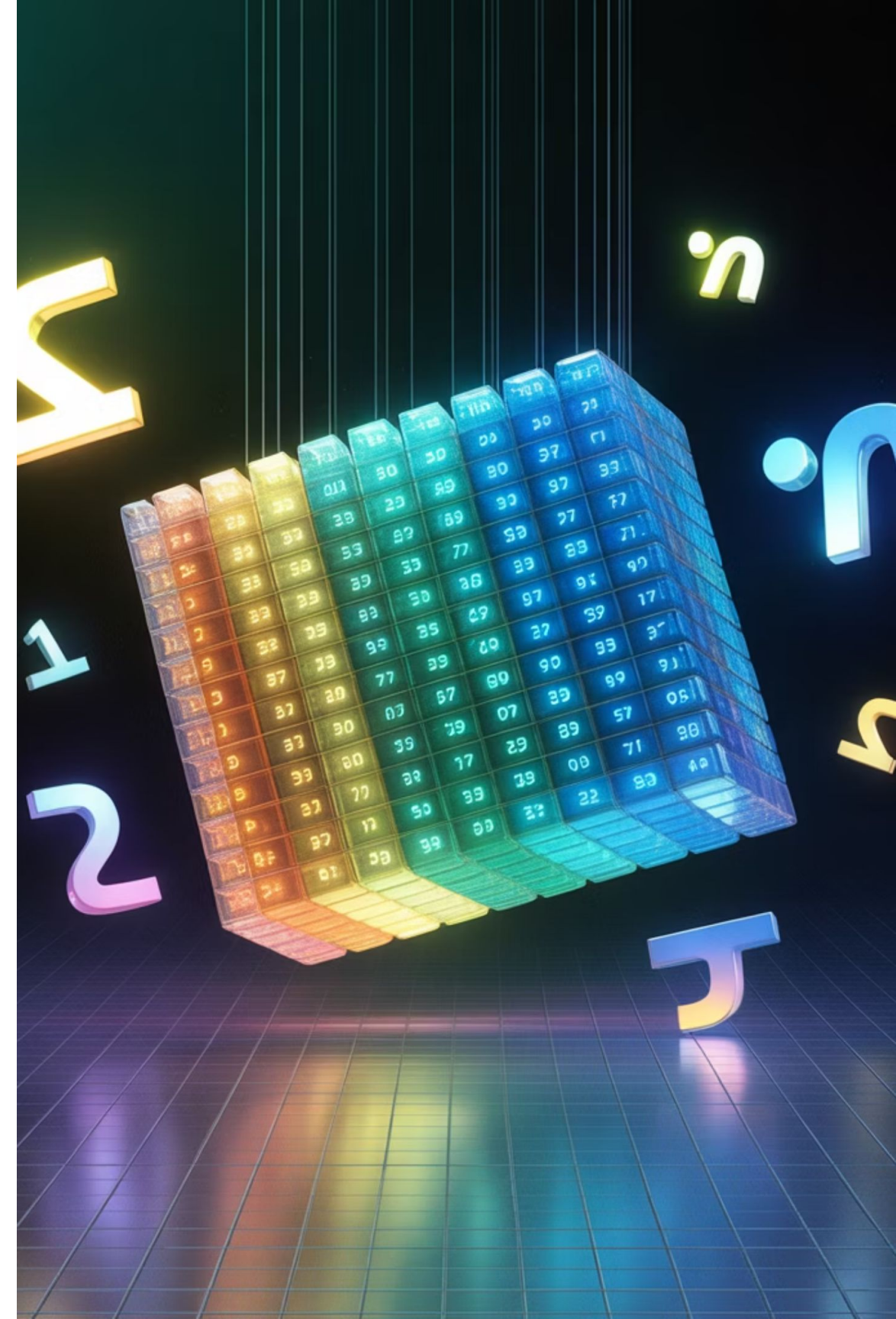


Exploración de propiedades básicas

Podemos consultar sus propiedades principales:

```
print(matriz.ndim) # Número de dimensiones → 2  
print(matriz.shape) # Forma → (2, 3): 2 filas, 3 columnas  
print(matriz.size) # Total de elementos → 6  
print(matriz.dtype) # Tipo de dato de los elementos → ej: int64
```

Estas propiedades son esenciales al analizar la compatibilidad con otras funciones o librerías de procesamiento.



Operaciones vectorizadas en matrices

Uno de los beneficios de NumPy es que las operaciones pueden aplicarse directamente sobre toda la matriz, sin bucles:

```
print(matriz * 10) # [[10 20 30]# [40 50 60]]
```

También puedes realizar operaciones más complejas como:

```
print(matriz + np.array([[10, 10, 10], [0, 0, 0]]))
```

Este comportamiento es posible gracias a la difusión de valores (broadcasting) que NumPy aplica de forma automática, y que exploraremos más adelante.

Acceso a elementos

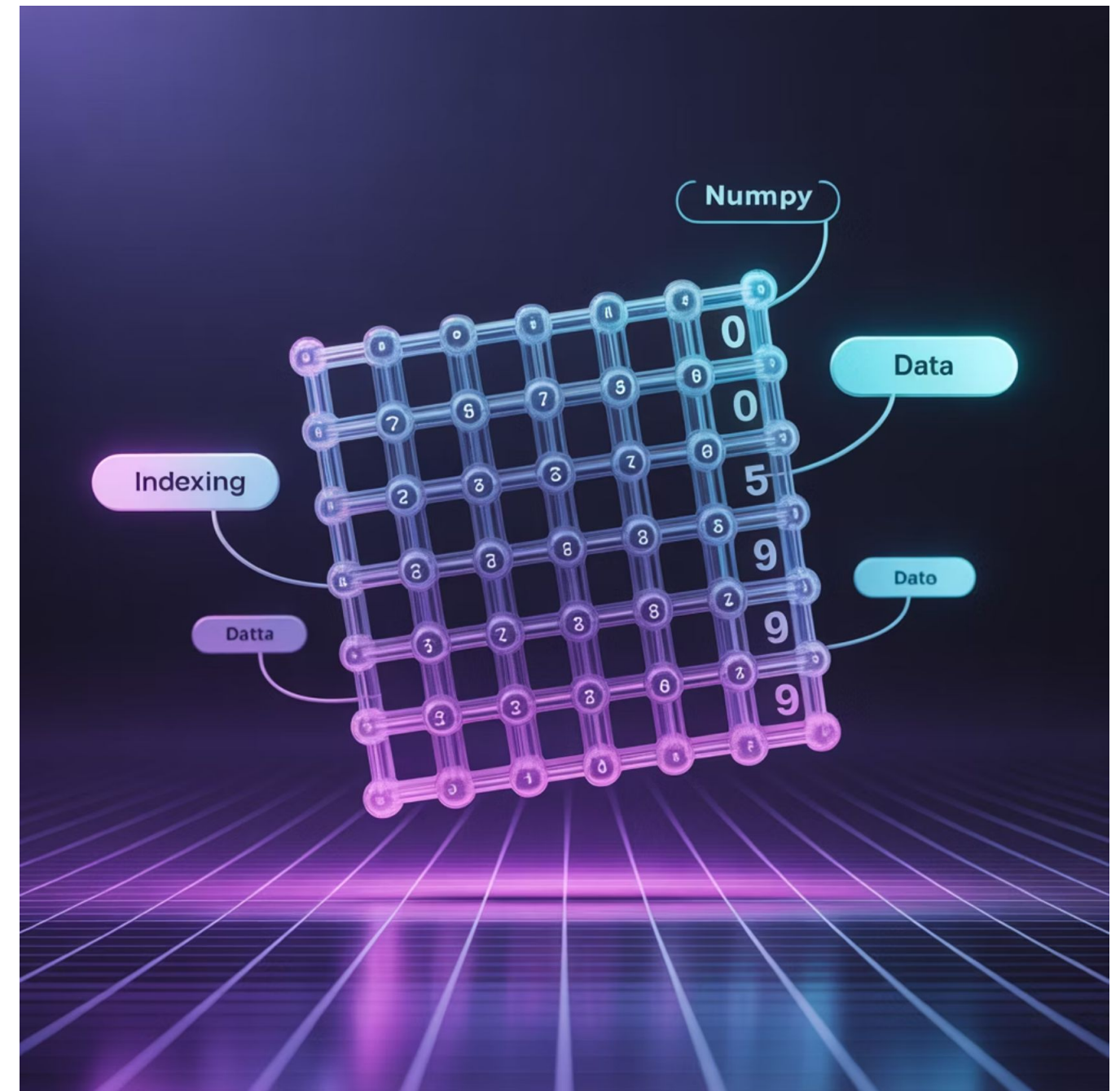
Al igual que con vectores, puedes acceder a valores específicos:

```
print(matriz[0, 1]) # Fila 0, Columna 1 → 2
```

También puedes acceder a una fila o columna completa:

```
print(matriz[0]) # Fila completa: [1 2 3]  
print(matriz[:, 1]) # Columna 1: [2 5]
```

Esta capacidad de indexación es fundamental en manipulación y limpieza de datos.



Uso de matrices en análisis



Machine learning

Entradas para modelos supervisados (X) y etiquetas (y).



Imágenes

Cada píxel es representado como una matriz de valores.



Series temporales multivariadas

Una fila por instante, columnas por variable.

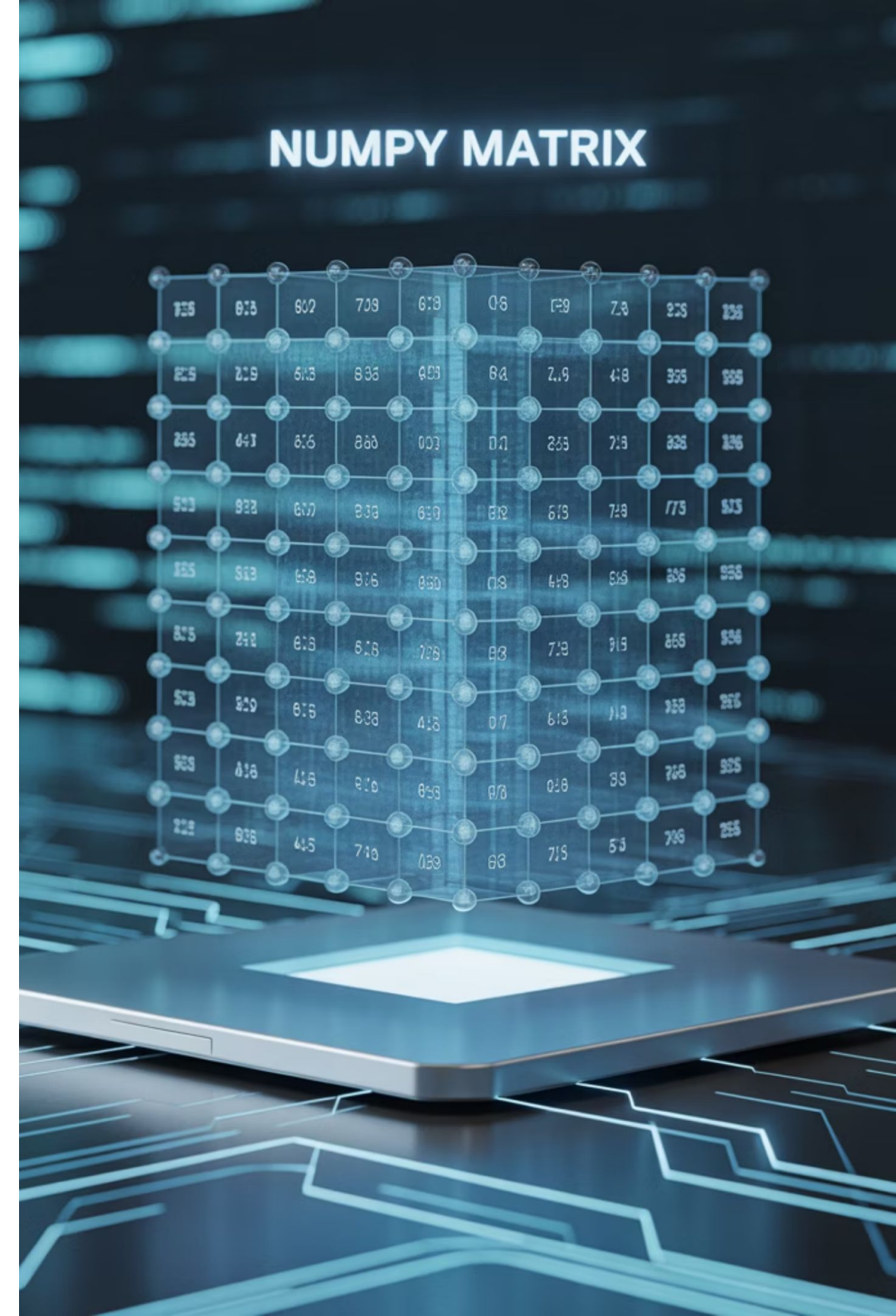


Álgebra lineal

Para aplicar transformaciones, resolver ecuaciones o realizar descomposiciones.

Recomendaciones al trabajar con matrices

- Siempre valida la forma de la matriz con `.shape` antes de realizar operaciones entre múltiples arreglos.
- Asegúrate de que las dimensiones coincidan para realizar operaciones aritméticas o lógicas.
- Para matrices muy grandes, considera el uso de estructuras dispersas si hay muchos ceros (ver SciPy).



**`/* Generación de secuencias
automáticas con arange()*/`**

Generación de secuencias automáticas con `arange()`

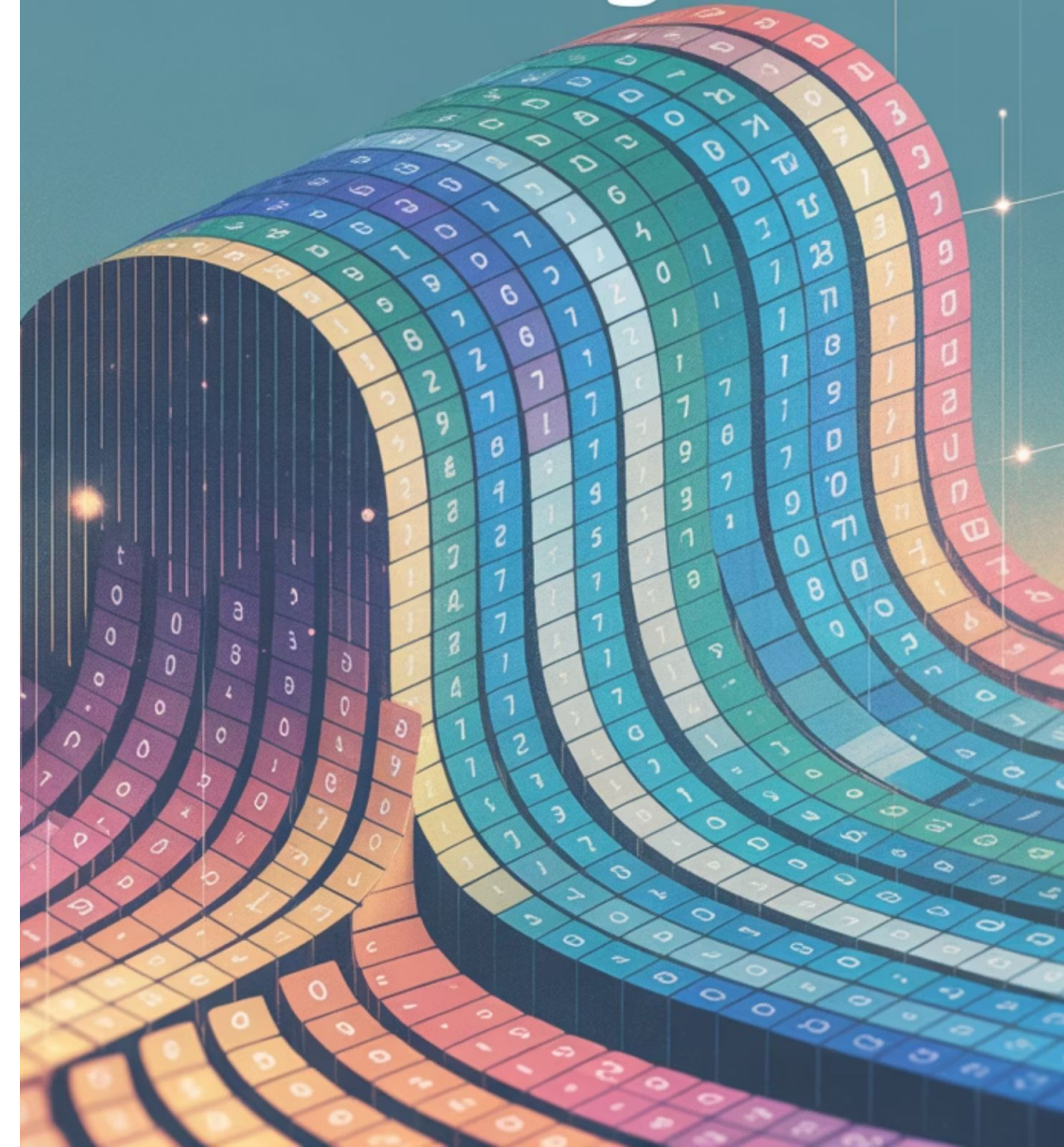
¿Cómo puedes crear fácilmente una secuencia numérica sin escribir cada valor manualmente?

En muchos contextos de ciencia de datos, simulaciones, análisis temporal o pruebas automatizadas, necesitamos construir secuencias numéricas.

Escribir cada valor manualmente sería ineficiente, especialmente cuando el rango es amplio o el paso es variable.

Para esto, NumPy ofrece la función `np.arange()` que permite crear vectores numéricos con gran flexibilidad.

numpy
arange



Sintaxis de np.arange()

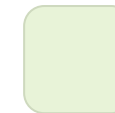
```
np.arange(inicio, fin, paso)
```



inicio: valor desde donde comienza la secuencia (incluyente).



fin: valor donde termina la secuencia (excluyente).



paso: valor por el que se incrementa (o decrementa) la secuencia.

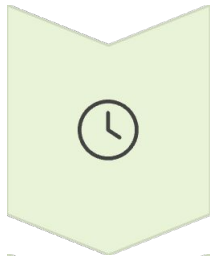
Ejemplo básico:

```
import numpy as np
rango = np.arange(0, 20, 5)
print(rango) # Resultado: [ 0 5 10 15 ]
```

En este caso, se genera una secuencia que parte en 0, se incrementa de 5 en 5, y se detiene antes de llegar a 20.

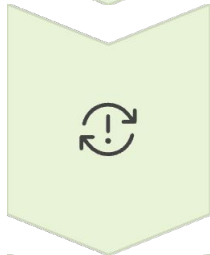
Aplicaciones típicas

Este tipo de arreglo es útil en tareas como:



Ejes temporales

Por ejemplo, segundos, minutos u horas.



Índices de simulación

Ciclos de prueba o iteraciones numéricas.



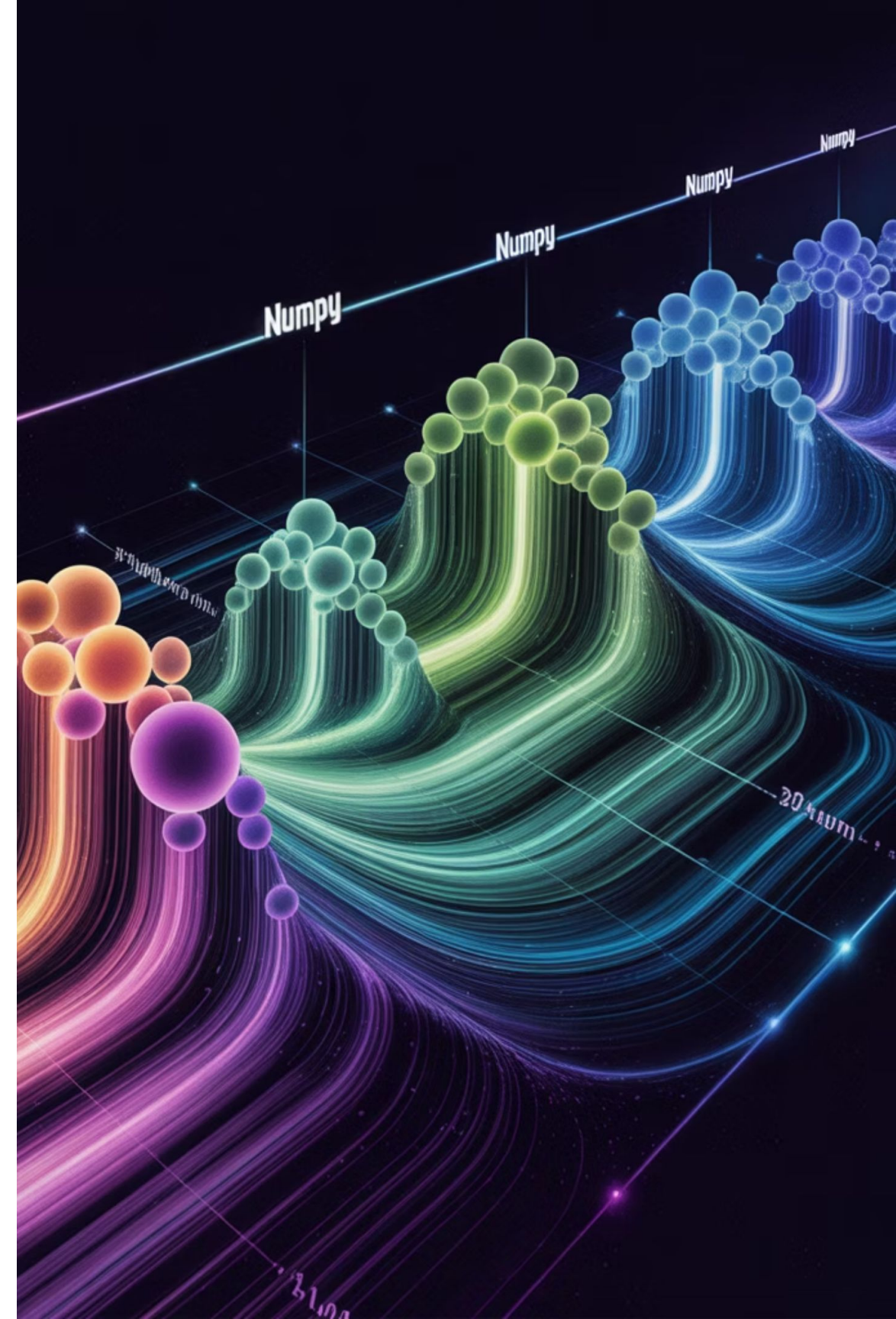
Marcadores de posición

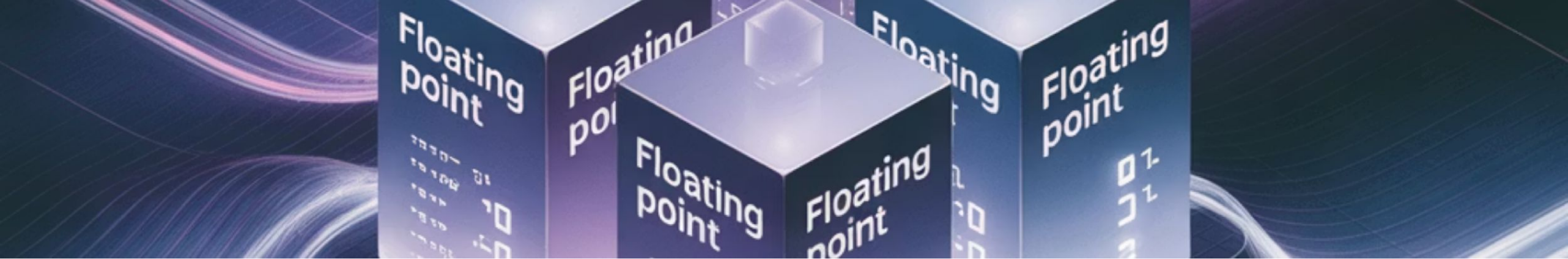
Plantillas de columnas o estructuras dummy.

Ejemplo aplicado:

```
# Crear una secuencia de minutos del 0 al 9  
tiempo = np.arange(0, 10, 1)  
print(tiempo) # Resultado: [0 1 2 3 4 5 6 7 8 9]
```

Esta construcción puede ser la base para registrar eventos cada minuto, simular lecturas horarias, o generar columnas base en modelos de series temporales.





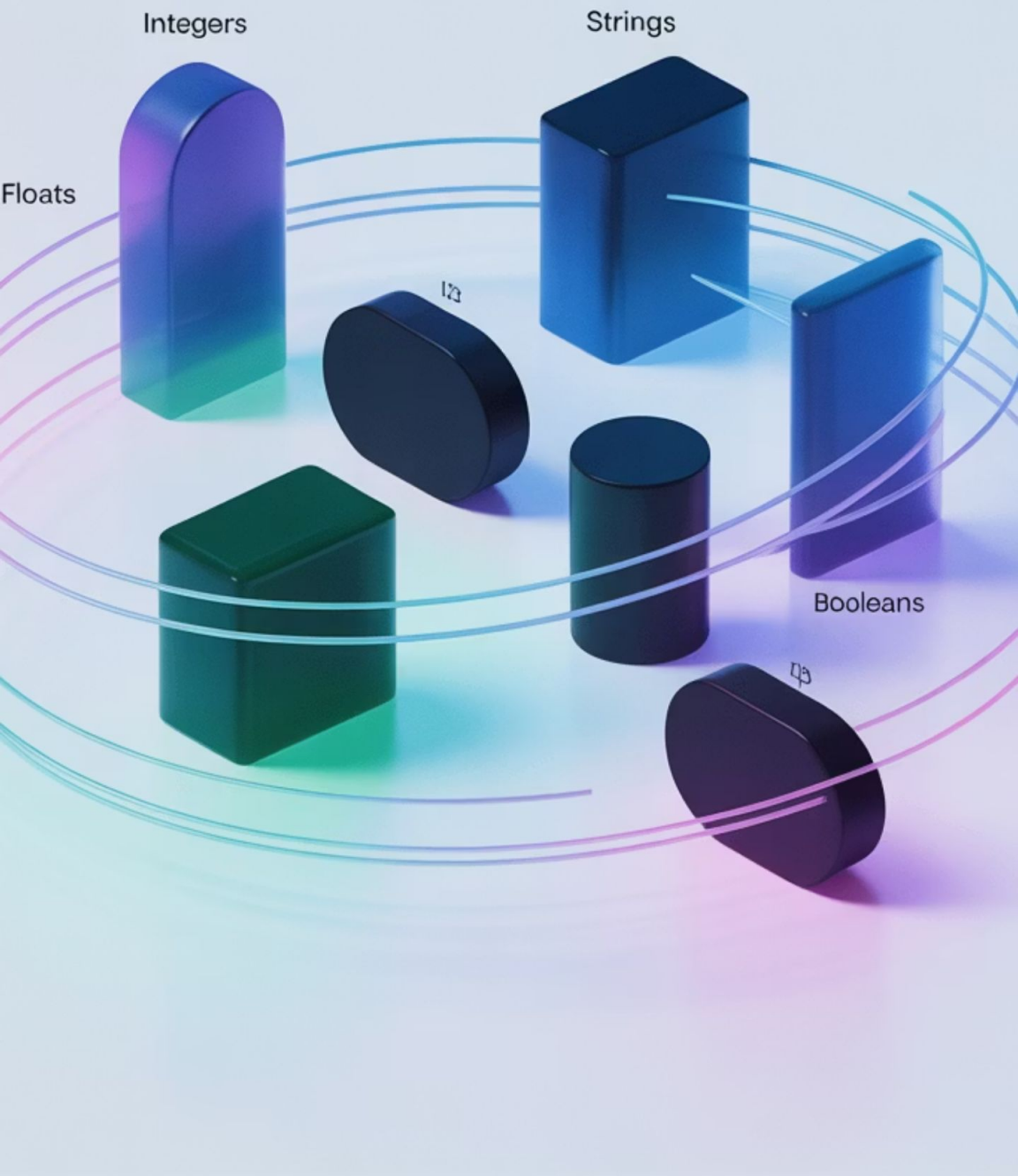
Uso con decimales

También se puede utilizar `np.arange()` con pasos decimales, aunque con precaución:

```
fracciones = np.arange(0, 1, 0.2)
print(fracciones)
# Resultado: [0. 0.2 0.4 0.6 0.8]
```

¡Importante! El uso de pasos flotantes puede generar errores de precisión debido al manejo interno de punto flotante en computación. Para evitar esto en contextos críticos, se recomienda `np.linspace()` (que permite definir el número exacto de puntos) y se revisará en sesiones posteriores.

Numpy Data Types



Verificación de tipo de datos

El tipo de datos resultante dependerá de los valores entregados:

```
print(np.arange(0, 5, 1).dtype) # int64  
print(np.arange(0, 1, 0.1).dtype) # float64
```

Esto es importante si se quiere controlar explícitamente el tipo de dato en cálculos posteriores.

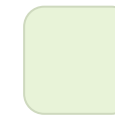
Buenas prácticas



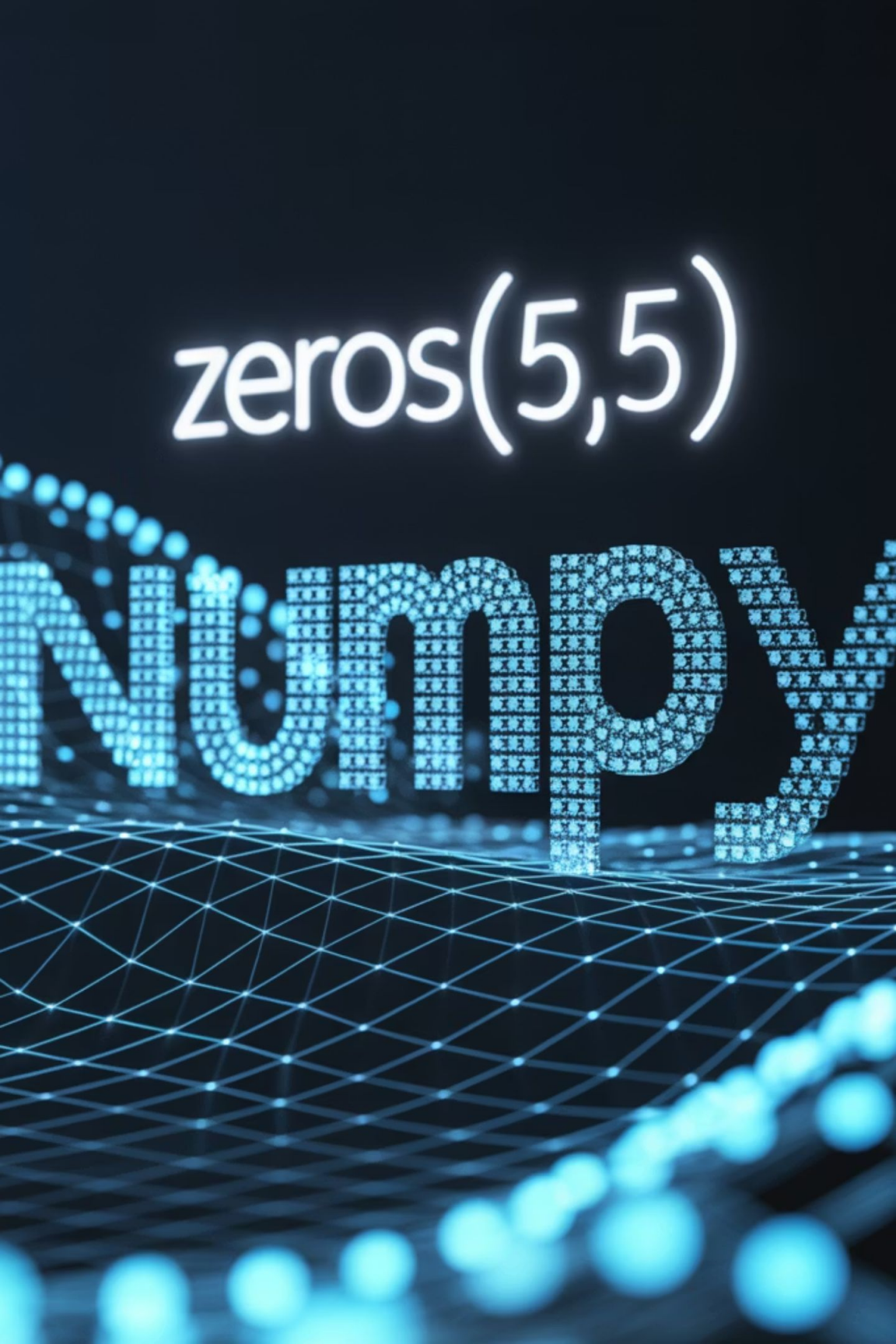
Siempre revisar el contenido generado: `print(rango)` o `print(rango.shape)`



No asumir que el valor final será incluido: `np.arange(0, 5, 1)` genera `[0 1 2 3 4]`, no incluye el 5.



Verificar el tipo de datos si se hará análisis numérico preciso (por ejemplo, estadísticas, modelos, comparaciones).



`zeros(5,5)`

Matrices de ceros con `np.zeros()`

¿Cómo crear una estructura base completamente vacía para luego llenarla con datos reales?

En múltiples aplicaciones de ciencia de datos, análisis numérico o programación científica, es común comenzar con una estructura vacía que se llenará con información posteriormente. Estas estructuras iniciales permiten definir la forma del problema antes de contar con los datos. Para esto, NumPy ofrece la función `np.zeros()`, que permite crear arreglos donde todos los elementos tienen el valor 0.

Sintaxis básica

```
np.zeros(formato, dtype=float)
```



formato: tupla que define la forma del arreglo (filas, columnas).



dtype (opcional): tipo de dato del arreglo (por defecto, float).

Ejemplo: matriz de ceros

```
import numpy as np  
plantilla = np.zeros((3, 4))  
print(plantilla)
```

Resultado:

```
[[0. 0. 0. 0.] [0. 0. 0. 0.] [0. 0. 0. 0.]]
```

Este código crea una matriz de 3 filas por 4 columnas, ideal para almacenar datos en una tabla de observaciones aún no completadas.



Vectores y matrices de unos con `np.ones()`

¿En qué situaciones es útil comenzar con una estructura donde todos los valores son 1?

En ciencia de datos y computación numérica, hay escenarios donde se necesita partir con una estructura homogénea con valores iguales a 1. Esto es útil para representar:



Máscaras binarias (por ejemplo, "todo activado"),



Estados iniciales de un sistema antes del procesamiento,



Matrices de normalización o escalado,



Constantes de calibración o ponderaciones fijas.

Para esto, NumPy proporciona la función `np.ones()`, que genera arreglos completamente poblados con el valor 1.

Creación de un vector de unos

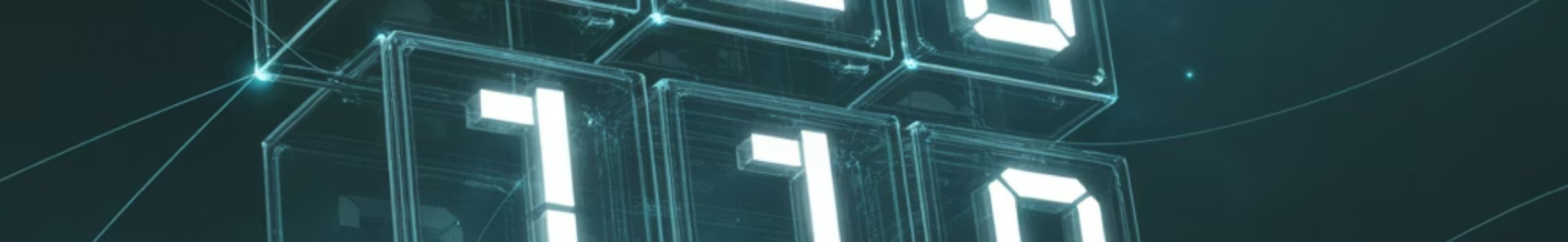
```
import numpy as np  
unos = np.ones(5)  
print(unos)
```

Resultado:

```
[1.  1.  1.  1.  1.]
```

Esto genera un vector unidimensional de cinco elementos, todos con el valor 1.0. Por defecto, los datos son de tipo float64.





Creación de una matriz de unos

```
estructura = np.ones((2, 3))  
print(estructura)
```

Resultado:

```
[[1.  1.  1.] [1.  1.  1.]]
```

Esta matriz de 2 filas por 3 columnas puede utilizarse, por ejemplo, para simular un conjunto de observaciones iniciales idénticas.

Multiplicación por un escalar

Si se necesita una estructura homogénea con otro valor, basta multiplicar el resultado por el número deseado:

```
np.ones((2, 3)) * 100
```

Resultado:

```
[[100. 100. 100.] [100. 100. 100.]]
```

Esto es útil para definir condiciones iniciales estables, constantes de prueba o patrones repetitivos.

Cambiar tipo de datos (dtype)

Por defecto, `np.ones()` genera `float64`, pero puede especificarse otro tipo de datos:

```
unos_int = np.ones((3, 3), dtype=int)  
print(unos_int)
```

Resultado:

```
[[1 1 1] [1 1 1] [1 1 1]]
```

Esto es relevante cuando se requieren operaciones discretas o ahorro de memoria.

Uso en análisis y modelos

- En regresiones lineales, se puede usar una columna de unos como término independiente (bias).
- En algoritmos iterativos, permite inicializar pesos o acumuladores.
- En evaluaciones condicionales, sirve como base para máscaras booleanas o simulaciones.

Comparación con otras funciones de inicialización

Función	Descripción	Ejemplo
np.zeros()	Arreglo lleno de ceros	np.zeros((2, 2))
np.ones()	Arreglo lleno de unos	np.ones((2, 2))
np.full()	Arreglo con un valor específico	np.full((2, 2), fill_value=7)

Esta comparación permite elegir con claridad la herramienta adecuada según el contexto de trabajo.

Ejercicio guiado- Simulación de estructuras de monitoreo ambiental



Ejercicio

Instrucciones

Objetivo: Aplicar la creación de vectores, matrices y funciones automáticas de generación utilizando NumPy, para estructurar datos simulados provenientes de sensores ambientales en un sistema de monitoreo urbano.

Situación: Trabajas en el área de desarrollo de un sistema automatizado de monitoreo de variables ambientales. Cada dispositivo desplegado en la ciudad registra lecturas continuas de temperatura, humedad y concentración de partículas. Tu jefatura te solicita generar una estructura inicial que sirva para simular, almacenar y procesar estos datos rápidamente en tiempo real, sin escribir cada dato manualmente ni usar estructuras poco eficientes como listas anidadas.

Además, te piden dejar lista una plantilla vacía para almacenar futuras mediciones, así como un vector que represente el estado inicial de calibración de todos los sensores.



Ejercicio

Instrucciones

Crear un vector con una lectura individual (3 variables: temperatura, humedad, partículas)

```
import numpy as np
lectura = np.array([22.5, 55.0, 83.1])
print("Lectura individual:", lectura)
```

¿Qué representa cada número? ¿Qué pasaría si agregamos una cuarta variable como presión?

Ejercicio

Instrucciones

Crear una matriz con lecturas múltiples (4 registros de sensores)

Cada fila representa una medición completa en un instante de tiempo.

```
lecturas = np.array([[22.5, 55.0, 83.1],[23.0, 54.5, 80.2],[21.8, 56.2, 84.7],[22.3, 55.8, 81.4]])  
print("Lecturas múltiples:", lecturas)
```

¿Cómo puedes identificar la lectura con mayor nivel de partículas?

Ejercicio

Instrucciones

Crear una secuencia de tiempo que acompañe las mediciones

```
tiempo = np.arange(0, 4, 1)
print("Tiempos de lectura (minutos):", tiempo)
```

¿Y si los sensores midieran cada 15 segundos en vez de cada minuto?

¿Qué cambiaría?



Ejercicio

Instrucciones

Crear una plantilla vacía para almacenar futuras mediciones

Estructura de 10 filas (futuras observaciones) y 3 columnas (variables ambientales).

```
plantilla = np.zeros((10, 3))  
print("Plantilla vacía para nuevas mediciones:", plantilla)
```

¿Por qué es útil tener estructuras vacías antes de que lleguen los datos reales?



Ejercicio

Instrucciones

Crear un vector de calibración con valores iniciales iguales a 1

```
calibracion = np.ones(3)
print("Vector de calibración inicial:", calibracion)
```

¿Qué función cumple esta calibración en el monitoreo? ¿Cómo la modificarías en terreno?



Preguntas de reflexión y cierre

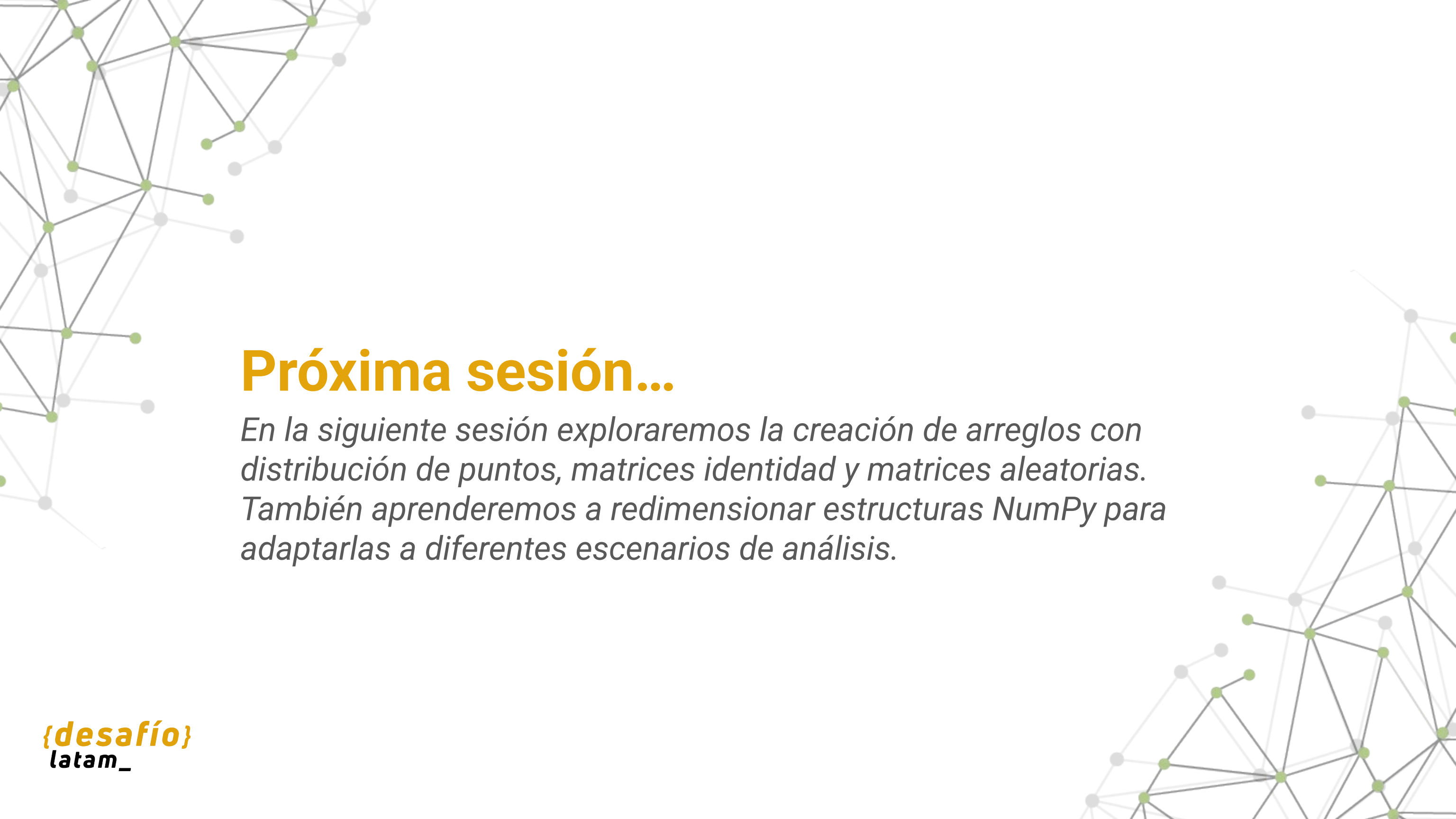
- ¿Qué ventajas tiene construir estos datos automáticamente en vez de definirlos uno a uno?
- ¿Cómo te ayuda NumPy a simular estructuras más cercanas a un caso real?
- ¿Qué tipo de operaciones podrías aplicar sobre estas estructuras en una etapa posterior del análisis?
- ¿Qué problemas podrías enfrentar si en lugar de usar arreglos NumPy usaras listas anidadas en Python?

Resumen

 Conocimos la biblioteca NumPy y su importancia para trabajar con estructuras numéricas en ciencia de datos.

 Aprendimos a crear vectores y matrices como formas eficientes de representar datos unidimensionales y bidimensionales.

 Usamos funciones automáticas de creación como `np.arange()`, `np.zeros()` y `np.ones()` para generar datos de prueba y estructuras base reutilizables.



Próxima sesión...

En la siguiente sesión exploraremos la creación de arreglos con distribución de puntos, matrices identidad y matrices aleatorias. También aprenderemos a redimensionar estructuras NumPy para adaptarlas a diferentes escenarios de análisis.